
EEEM071 Advanced Topics in Computer Vision and Deep Learning

Lab Week 7

Data Pre-processing and Augmentation

Dr. Xiatian Zhu

xiatian.zhu@surrey.ac.uk

1 Introduction

Today we learn how to use the `torchvision` package to help us do *data pre-processing and augmentation*. We will also learn how to modify the code section of the data loading in the coursework and the Fashion-MNIST for data augmentation. After modifying it, try run the experiments and observe the performance changes. Note that it might cause slow convergence due to slowing the overfitting process, but most of the time the final performance can be higher (if it is lower, try tuning the learning rate, if not, the augmentation might be too strong/too weak).

2 Torchvision package

`torchvision` is a popular Python package that provides access to datasets, models, and transforms for working with images and videos using PyTorch. It is a part of the PyTorch project, which is an open-source machine learning library that is widely used for building deep learning models.

`torchvision` provides a wide range of pre-built datasets, such as CIFAR-10, CIFAR-100, MNIST, and ImageNet, that can be used for training and testing image classification models. It also provides a number of pre-trained models, such as ResNet, VGG, and AlexNet, that can be used as a starting point for building more complex models.

In addition to datasets and models, `torchvision` also provides a set of image and video transforms that can be used to augment and preprocess images before feeding them into a machine learning model. These transforms include resizing, cropping, flipping, rotating, and normalizing images, among others.

Overall, `torchvision` makes it easier for developers to work with image and video data using PyTorch, and provides a high-level interface for common tasks such as loading data, preprocessing data, and building models.

3 Data Augmentation

Data augmentation is a technique used in machine learning and computer vision to artificially increase the size of a dataset by creating new, modified versions of existing data.

The process involves applying a set of transformations or modifications to the existing data, such as rotating, flipping, cropping, scaling, adding noise, or changing the color balance. These modifications create new variations of the original data, which can help to improve the performance of machine learning models by providing more diverse and representative training examples.

Data augmentation can be particularly useful in scenarios where the size of the available dataset is limited or where the data is unbalanced or biased. By creating additional examples that are similar to the original data but with small variations, data augmentation can help to prevent overfitting and improve the generalization ability of machine learning models.

4 Types of data augmentation

There are many different types of data augmentation that can be applied to images, but some of the most common ones include:

1. Rotation: Rotating the image by a certain angle, such as 90 degrees or 180 degrees.
2. Flipping: Flipping the image horizontally or vertically.
3. Scaling: Scaling the image up or down by a certain factor.
4. Cropping: Cropping a section of the image.
5. Translation: Shifting the image horizontally or vertically by a certain amount.
6. Adding noise: Adding random noise to the image to simulate real-world variations.
7. Color jitter: Adjusting the brightness, contrast, saturation, or hue of the image.
8. Elastic transformation: Distorting the image by applying random elastic deformations.

By applying one or a combination of these transformations to an image, we can generate a new, slightly modified version of the original image that can be used as a new training example. By applying these augmentations to a large number of images, we can significantly increase the size of our dataset and improve the generalization ability of our machine learning models.

5 Show me the code

For a list of available augmentation provided by torchvision, you can refer to this [link](#). Usually, we refer to the preprocessing (which also included data augmentation) of training set as `train_transform` and that of testing set as `test_transform`.

In the code section shown in Figure 1, we can modify the train transform by referring to the list in this [link](#). The original train transform shows the image in Figure 2. We simply add a `RandomCrop` augmentation into the `train_transform` as shown in Figure 3 and the resultant image is shown in Figure 4, note the differences.

Note `transforms.Compose` is to apply a series of transformation functions *sequentially* to the input image. Commonly, we start with `transforms.Resize(...)` and end with `transforms.ToTensor()` and some time with normalization `transforms.Normalize(...)`.

Now you can try different transformations/augmentations in this [link](#). Note FashionMNIST is a grayscale data so the color channel is 1, some augmentation techniques that involve multiple color channels might pop up errors. You can use CIFAR10 to see the effects of the different augmentation techniques, see Figure

```

# Setup the training data. Here we will use PyTorch's own build in functions to facilitate downloading the dataset

train_data = datasets.FashionMNIST(
    root="data", # where to download data to?
    train=True, # do we want the training dataset?
    download=True, # do we want to download yes/no?
    transform=torchvision.transforms.ToTensor(), # how do we want to transform the data?
    target_transform=None, # how do we want to transform the labels/targets?
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=torchvision.transforms.ToTensor(),
    target_transform=None
)

```

Figure 1: The code section to control train transform

5 to change the dataset from FashionMNIST to CIFAR10. Note that CIFAR10 is a built-in dataset in torchvision so we can change from FashionMNIST to CIFAR10 simply by editing one line. See more built-in datasets in this [link](#).

In the coursework code base, you can modify the train transform in `src/transforms.py/build_transforms` (see Figure 8). You are encouraged to modify the train transform and see whether your modifications can improve performance. You can mention your modifications and observations in your final report. Please refer to the Sec. 8.2 of CourseWork document for more requirements.

Note that you will see the test transform is always a simple resize and center crop, it is because we do not want any augmentation for testing but only the raw inputs.

There are many advanced augmentation techniques listed in [torchvision](#), try them yourself!



Figure 2: Visualization of the images.

```

# Setup the training data. Here we will use PyTorch's own build in functions to facilitate downloading the dataset

train_data = datasets.FashionMNIST(
    root="data", # where to download data to?
    train=True, # do we want the training dataset?
    download=True, # do we want to download yes/no?
    transform=transforms.Compose([transforms.RandomCrop(28, padding=4), transforms.ToTensor()]),
    # transform=torchvision.transforms.ToTensor(), # how do we want to transform the data?
    target_transform=None, # how do we want to transform the labels/targets?
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=torchvision.transforms.ToTensor(),
    target_transform=None
)

```

Figure 3: Modify the original train transform.

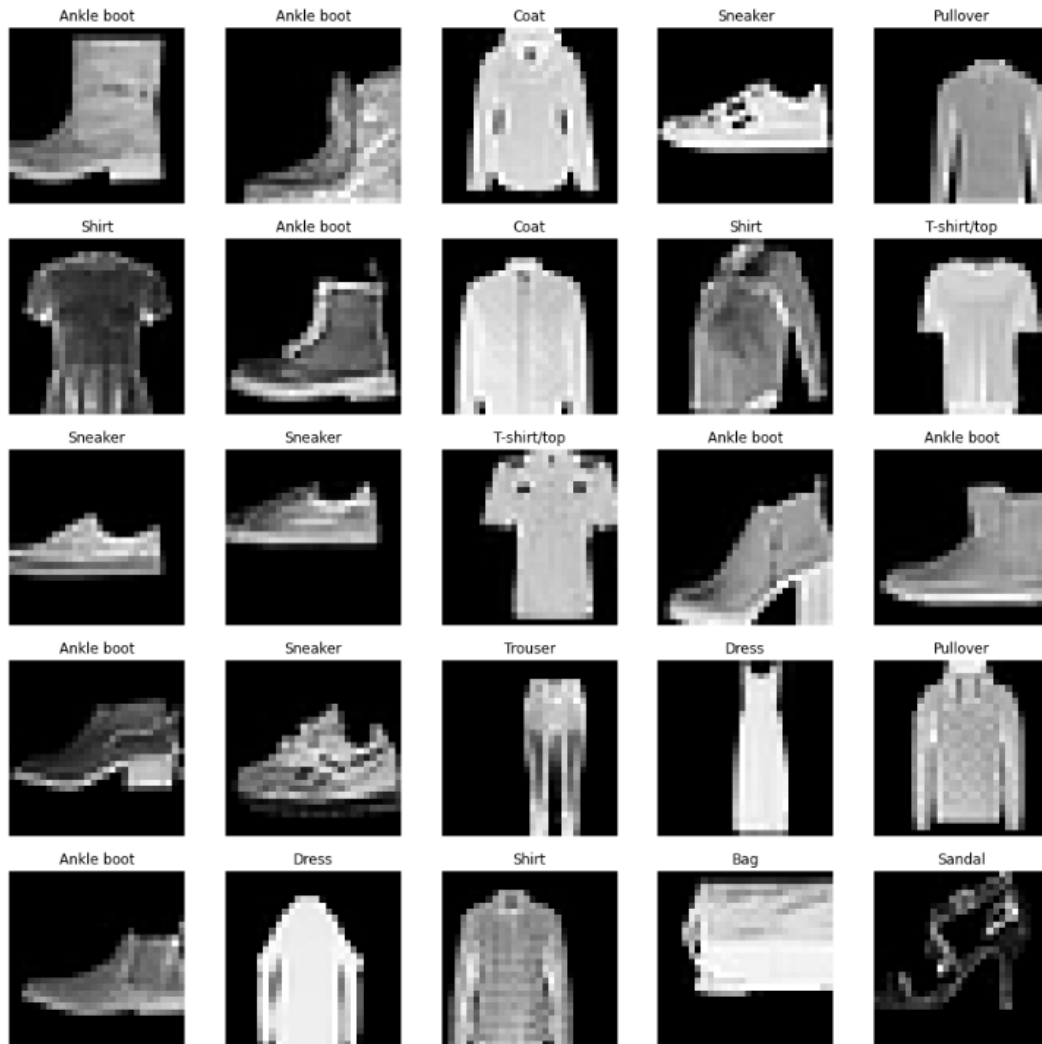


Figure 4: Visualization of the images with augmentation.

```
may the force be with you:

# Setup the training data. Here we will use PyTorch's own build in functions to facilitate downloading the dataset

train_data = datasets.CIFAR10(
    root="data", # where to download data to?
    train=True, # do we want the training dataset?
    download=True, # do we want to download yes/no?
    transform=transforms.Compose([transforms.RandomCrop(32, padding=4), transforms.ToTensor()]),
    # transform=torchvision.transforms.ToTensor(), # how do we want to transform the data?
    target_transform=None, # how do we want to transform the labels/targets?
)

test_data = datasets.CIFAR10(
    root="data",
    train=False,
    download=True,
    transform=torchvision.transforms.ToTensor(),
    target_transform=None
)
```

... Downloading <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz> to data/cifar-10-python.tar.gz
9% 15630336/170498071 [00:00<00:04, 37572276.65it/s]

Figure 5: Change FashionMNIST to CIFAR10.

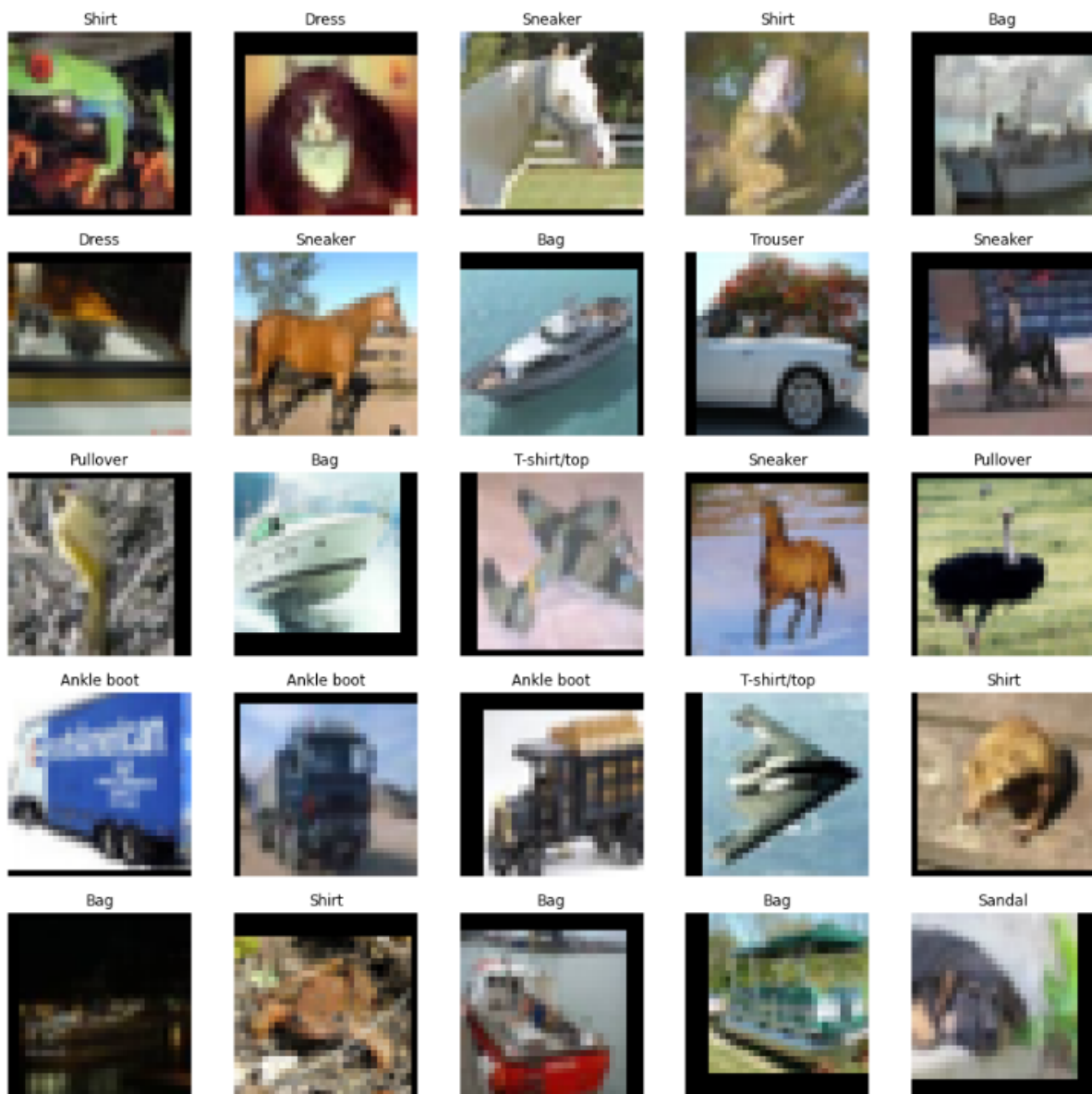


Figure 6: Example image of CIFAR10 (with RandomCrop augmentation).

✓
4s



```
# plot more images
torch.manual_seed(42)
torch.cuda.manual_seed(42)

fig = plt.figure(figsize=(16,16))
rows, cols = 5, 5
for i in range(1, rows*cols+1):
    random_idx = torch.randint(0, len(train_data), size=[1]).item()
    img, label = train_data[random_idx]
    fig.add_subplot(rows, cols, i);
    # plt.imshow(img.squeeze(), cmap="gray")
    plt.imshow(img.permute(1, 2, 0))
    plt.title(class_names[label])
    plt.axis(False);
```

Figure 7: How to fix the display error. It is because `plt.imshow` accepts the image of (H, W, C) to display, but PyTorch processes images in terms of (C, H, W), therefore we need to permute the data dimension using `permute`. This means change the original data dimension C(0), H(1), W(2) to H(1), W(2), C(0).

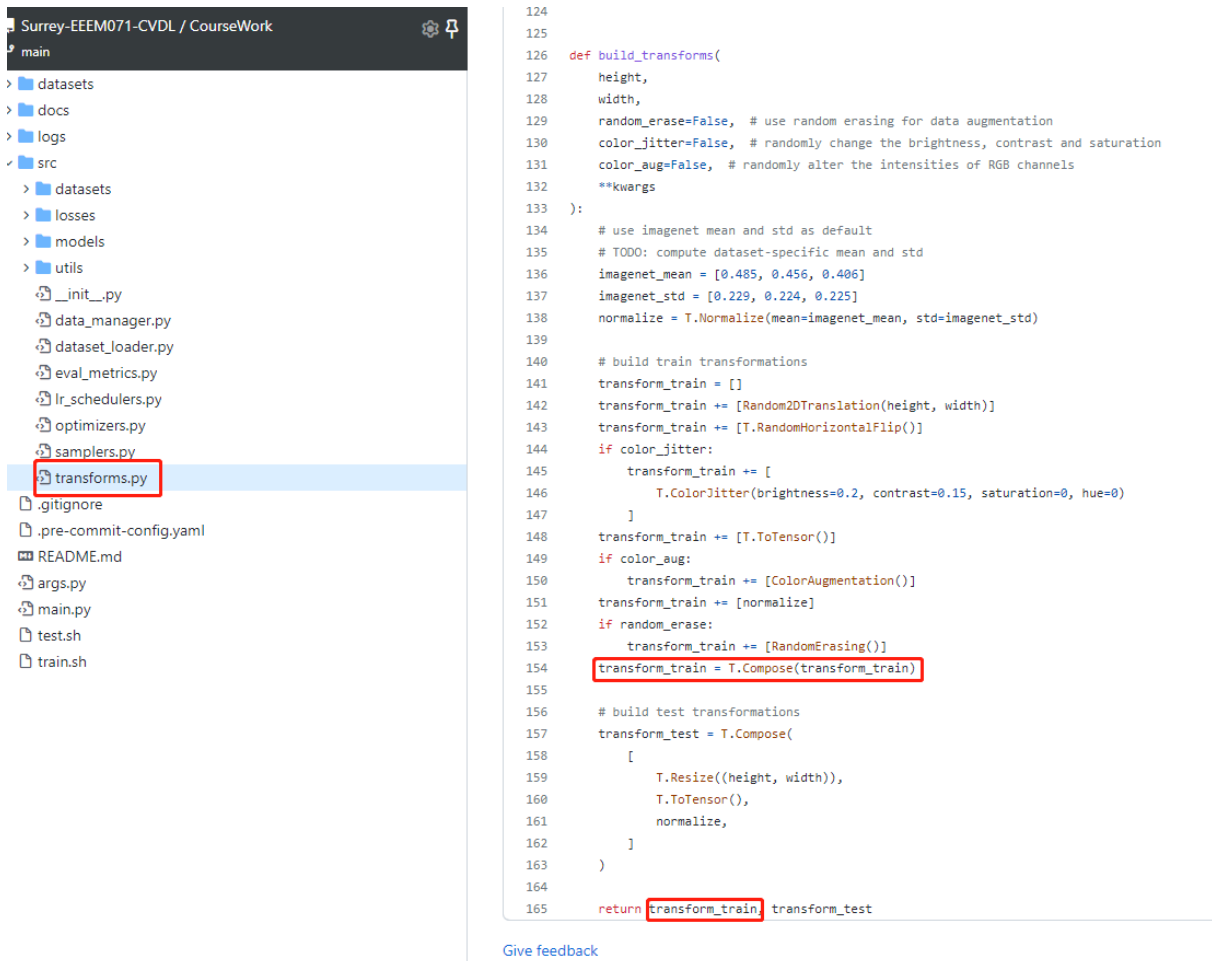


Figure 8: Try modifying the train transform and see whether it can improve the performance.